

Universidade Federal de São Carlos - UFSCar
Centro de Ciências Exatas e de Tecnologia
Departamento de Computação

Relatório de Prática: Cálculo de PI via Método de Monte Carlo com FreeRTOS

Gabriel Mello Silveira Targas - 812990
Guilherme Barbosa - 811692

Professor: Dr. Emerson Carlos Pedrino

São Carlos, SP
2026

Sumário

1	Introdução e Objetivo	2
2	Embasamento Teórico	2
2.1	Método de Monte Carlo para Cálculo de PI	2
2.2	FreeRTOS em Microcontroladores Single-Core	2
3	Desenvolvimento - Implementação A (2 Tarefas)	2
3.1	Configuração Inicial e Instanciação (Setup)	2
3.2	Lógica da Tarefa de Cálculo	3
3.3	Lógica da Tarefa de Impressão	3
4	Desenvolvimento - Implementação B (4 Tarefas)	4
4.1	O Problema da Memória e o Stack Overflow	4
4.2	Otimização e Lógica das Múltiplas Tarefas	5
5	Comparação e Benchmark (Implementação A vs B)	5
6	Conclusão	6
7	Referências	6

1 Introdução e Objetivo

Este relatório documenta a implementação do cálculo da constante Pi (π) através do método estocástico de Monte Carlo, embarcado em um microcontrolador Arduino Uno (ATmega328P). Para a gestão da concorrência e escalonamento dos processos matemáticos e de comunicação, utilizou-se o sistema operacional de tempo real **FreeRTOS**.

O objetivo principal deste estudo é desenvolver e analisar duas arquiteturas distintas de *software*: a primeira (**Implementação A**) utilizando duas tarefas (uma para cálculo e outra para exibição), e a segunda (**Implementação B**) utilizando quatro tarefas (três para cálculo concorrente e uma para exibição). Ao final, um *benchmark* comparativo é apresentado para discutir os impactos do escalonamento em sistemas operacionais *single-core* sob cargas de trabalho limitadas por CPU (*CPU-bound*).

2 Embasamento Teórico

2.1 Método de Monte Carlo para Cálculo de PI

O método de Monte Carlo baseia-se na geração de números pseudo-aleatórios para aproximar resultados matemáticos complexos. Para estimar o valor de π , simula-se um quadrante de um círculo de raio $r = 1$ inscrito em um quadrado de lado 1. Ao gerar pontos aleatórios com coordenadas (x, y) entre 0 e 1, verifica-se se o ponto cai dentro do círculo através da equação geométrica: $x^2 + y^2 \leq 1$. A razão entre os pontos que caem dentro do círculo e o total de pontos gerados aproxima-se da área do quarto de círculo ($\frac{\pi}{4}$). Multiplicando esta razão por 4, obtém-se o valor de π .

2.2 FreeRTOS em Microcontroladores Single-Core

O ATmega328P é um processador de núcleo único, ou seja, não possui paralelismo real em *hardware*. O FreeRTOS cria a ilusão de execução simultânea alternando rapidamente o uso da CPU entre as tarefas (*Context Switching*). As tarefas obedecem a uma hierarquia estrita de prioridades: a tarefa pronta com maior prioridade sempre assume o controle da CPU. Para evitar o fenômeno de *starvation* (onde tarefas de menor prioridade nunca são executadas), as tarefas de alta prioridade devem ceder o processamento periodicamente utilizando funções de bloqueio como `vTaskDelay()`.

3 Desenvolvimento - Implementação A (2 Tarefas)

Nesta etapa inicial, instanciou-se o sistema com o mínimo necessário para a comprovação do conceito operando em tempo real, utilizando apenas uma tarefa para cálculo e outra para exibição.

3.1 Configuração Inicial e Instanciação (Setup)

Na rotina de `setup`, inicializamos a comunicação serial e declaramos o semáforo do tipo *Mutex* para proteger as variáveis globais `total_points` e `inside_circle` de condições de corrida. A prioridade da tarefa de cálculo foi definida como 2 (maior), e a de impressão como 1 (menor).

```

1 #include <Arduino_FreeRTOS.h>
2 #include <semphr.h>
3
4 volatile unsigned long total_points = 0;
5 volatile unsigned long inside_circle = 0;
6 SemaphoreHandle_t xMutex;
7
8 void setup() {
9     Serial.begin(115200);
10    while (!Serial);
11
12    randomSeed(analogRead(0));
13    xMutex = xSemaphoreCreateMutex();
14
15    if (xMutex != NULL) {
16        // Tarefa de Calculo: Prioridade 2 (Maior)
17        xTaskCreate(TaskCalcPI, "Calc", 128, NULL, 2, NULL);
18        // Tarefa de Print: Prioridade 1 (Menor)
19        xTaskCreate(TaskPrint, "Print", 192, NULL, 1, NULL);
20    }
21 }
22
23 void loop() {} // O FreeRTOS assume o controle do fluxo

```

3.2 Lógica da Tarefa de Cálculo

A TaskCalcPI é projetada para rodar em laço infinito. Para que a tarefa de impressão (de menor prioridade) não sofra *starvation*, o processamento matemático é fatiado em lotes de 200 iterações. Ao fim de cada lote, a tarefa toma posse do *Mutex*, atualiza os pontos globais de forma segura, e invoca um `vTaskDelay(1)`, cedendo 1 *tick* da CPU.

```

1 void TaskCalcPI(void *pvParameters) {
2     (void) pvParameters;
3     while (1) {
4         unsigned long local_total = 0;
5         unsigned long local_inside = 0;
6
7         // Processamento estocastico em lotes
8         for(int i = 0; i < 200; i++) {
9             float x = random(10000) / 10000.0;
10            float y = random(10000) / 10000.0;
11            if ((x * x) + (y * y) <= 1.0) local_inside++;
12            local_total++;
13        }
14
15        // Acesso as variaveis protegidas
16        if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
17            total_points += local_total;
18            inside_circle += local_inside;
19            xSemaphoreGive(xMutex);
20        }
21
22        // Libera a CPU intencionalmente
23        vTaskDelay(1);
24    }
25 }

```

3.3 Lógica da Tarefa de Impressão

A TaskPrint aproveita as janelas de ociosidade deixadas pelo `vTaskDelay` do cálculo. Ela lê as variáveis globais, processa a razão final multiplicada por 4 e envia o resultado para o monitor serial a cada 1 segundo (1000 ms).

```

1 void TaskPrint(void *pvParameters) {
2     (void) pvParameters;
3     while (1) {

```

```

4   unsigned long t = 0, i = 0;
5
6   // Leitura protegida
7   if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
8       t = total_points;
9       i = inside_circle;
10      xSemaphoreGive(xMutex);
11  }
12
13  // Calculo final de PI e envio serial
14  if (t > 0) {
15      float pi_calc = 4.0 * ((float)i / t);
16      Serial.print("[2 Tarefas] Pontos processados: ");
17      Serial.print(t);
18      Serial.print(" | Valor de PI: ");
19      Serial.println(pi_calc, 5);
20  }
21  vTaskDelay(pdMS_TO_TICKS(1000));
22  }
23 }

```

Como evidenciado na Figura 1, a arquitetura de apenas duas tarefas manteve a estabilidade do sistema operacional embarcado, gerando uma grande quantidade de iterações matemáticas sem comprometer o barramento serial.

```

[2 Tarefas] Pontos processados: 200000
[2 Tarefas] Pontos processados: 3400 | Valor de PI: 3.12235
[2 Tarefas] Pontos processados: 6600 | Valor de PI: 3.12545

```

Figura 1: Monitor serial da Implementação A (2 tarefas). O sistema opera de forma concisa e estável.

4 Desenvolvimento - Implementação B (4 Tarefas)

Nesta segunda abordagem, o objetivo foi instanciar múltiplas rotinas de cálculo para disputar o processamento matematicamente simultâneo.

4.1 O Problema da Memória e o Stack Overflow

A tentativa inicial de instanciar 3 tarefas de cálculo juntamente com a de impressão causou falhas severas no microcontrolador. Como o Arduino Uno possui apenas **2KB de RAM**, a soma do espaço de pilha (*Stack*) solicitado na função `xTaskCreate` estourou o limite físico da memória. O resultado foi a corrupção do *buffer* de comunicação, gerando caracteres incompreensíveis na tela, conforme registrado na Figura 2.

```

[4 Tarefas] Pontos processados: 6de PI: 3.12667
[4 Tarefas] Pontos processados: 4800 | Valor de PI: 3.12833
?}!y? ?P?????????6?DaHa?V???.

```

Figura 2: Evidência de Stack Overflow e corrupção de memória gerando artefatos (*lixo*) no monitor serial devido à sobrecarga das 4 tarefas.

4.2 Otimização e Lógica das Múltiplas Tarefas

Para viabilizar a Implementação B e cessar o *Stack Overflow*, o código precisou ser altamente otimizado. No `setup`, o tamanho do bloco de memória de cada tarefa foi reduzido. Na função de Impressão, utilizamos a **Macro F()** nos comandos `Serial.print()`, movendo os textos estáticos da RAM para a memória *Flash* do Arduino.

```
1 void setup() {
2   Serial.begin(115200);
3   while (!Serial);
4   randomSeed(analogRead(0));
5   xMutex = xSemaphoreCreateMutex();
6
7   if (xMutex != NULL) {
8     // Criacao de 3 threads de calculo com memoria reduzida (100)
9     xTaskCreate(TaskCalcPI, "C1", 100, (void*)1, 2, NULL);
10    xTaskCreate(TaskCalcPI, "C2", 100, (void*)2, 2, NULL);
11    xTaskCreate(TaskCalcPI, "C3", 100, (void*)3, 2, NULL);
12
13    // Tarefa de Print tambem com memoria reduzida (128)
14    xTaskCreate(TaskPrint, "Pr", 128, NULL, 1, NULL);
15  }
16 }
```

Além disso, na função de cálculo (`TaskCalcPI`), o lote de iterações por ciclo foi reduzido à metade (de 200 para 100) para garantir uma alternância ágil de *Round-Robin* entre as três rotinas matemáticas sem estrangular o núcleo de processamento.

```
1 void TaskCalcPI(void *pvParameters) {
2   while (1) {
3     unsigned long local_total = 0;
4     unsigned long local_inside = 0;
5
6     // Lote reduzido para 100 iteracoes
7     for(int i = 0; i < 100; i++) {
8       float x = random(10000) / 10000.0;
9       float y = random(10000) / 10000.0;
10      if ((x * x) + (y * y) <= 1.0) local_inside++;
11      local_total++;
12    }
13
14    if (xSemaphoreTake(xMutex, portMAX_DELAY) == pdTRUE) {
15      total_points += local_total;
16      inside_circle += local_inside;
17      xSemaphoreGive(xMutex);
18    }
19    vTaskDelay(1);
20  }
21 }
```

5 Comparação e Benchmark (Implementação A vs B)

A execução das duas rotinas otimizadas revelou resultados operacionais contra-intuitivos sob a lente de sistemas multicore. A **Implementação A (2 tarefas) processou uma quantidade consideravelmente maior de pontos por segundo** em comparação à **Implementação B (4 tarefas)**.

Os fatores técnicos de *hardware* e sistema operacional que justificam este gargalo no Arduino Uno incluem:

- **Custo de Troca de Contexto (Context Switching Overhead):** Sendo o microcontrolador *single-core*, ele não executa física e simultaneamente as 3 instâncias

de cálculo da Implementação B. O *scheduler* do FreeRTOS precisa paralisar rotineiramente a execução de uma tarefa, salvar todo o seu conjunto de registradores na memória, e carregar o conjunto da próxima. Este trabalho consome ciclos preciosos de *clock* que estariam sendo usados efetivamente para o algoritmo matemático na Implementação A.

- **Contenção do Mutex:** Com três tarefas disputando a região crítica simultaneamente na Implementação B, aumenta-se drasticamente o tempo passivo em que as *threads* ficam bloqueadas pela instrução `xSemaphoreTake`, ociosas à espera da liberação do recurso global.

6 Conclusão

A prática comprovou que a aplicação de arquiteturas orientadas a múltiplas tarefas (*multi-threading*) em microcontroladores de núcleo único exige uma criteriosa análise de *design* de *software*. Para cenários puramente dependentes do esforço da CPU (*CPU-bound*), como o cálculo geométrico intensivo e geração estocástica do método de Monte Carlo, particionar o trabalho eleva o custo de gerenciamento (*overhead*) a níveis deletérios.

Nesses sistemas, a implementação enxuta baseada em poucas instâncias (Implementação A) revelou-se drasticamente mais performática e imune à restrição severa de memória física. Demonstrou-se, portanto, que a utilidade central de um RTOS não reside na aceleração de blocos matemáticos brutos, mas sim em orquestrar de forma não-bloqueante sistemas com alta dependência de operações I/O e eventos assíncronos externos.

7 Referências

- [1] Slides de aula fornecidos pelo professor.
- [2] BARRY, Richard. *Mastering the FreeRTOS Real Time Kernel: A Hands-On Tutorial Guide*. Real Time Engineers Ltd., 2016. Documentação oficial do FreeRTOS.
- [3] MICROCHIP TECHNOLOGY INC. *ATmega328P 8-bit AVR Microcontroller Datasheet*. San Jose, CA, USA. (Referência para a arquitetura Single-Core e limite de 2KB de SRAM).
- [4] ARDUINO. *Arduino Language Reference* (Documentação sobre a macro `F()` e funções estocásticas). Disponível em: <<https://www.arduino.cc/reference/en/>>. Acesso em: 16 jun. 2026.
- [5] RUBINSTEIN, Reuven Y.; KROESE, Dirk P. *Simulation and the Monte Carlo Method*. 3. ed. John Wiley & Sons, 2016.